Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71200556**
Nama Lengkap	**Frans Sailapra**
Minggu ke / Materi	**06 / Struktur Kontrol Perulangan**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 1 : Definisi Perulangan

Dalam pemrograman, jalannya suatu program dapat diatur menggunakan struktur kontrol. Struktur kontrol ini berfungsi untuk menentukan bagaimana suatu program dieksekusi oleh komputer. Secara umum terdapat tiga jenis struktur kontrol dalam pemrograman, yaitu sekuensial, percabangan, dan perulangan.

Struktur kontrol sekuensial merupakan bentuk eksekusi program yang paling sederhana, di mana setiap perintah dijalankan secara berurutan dari atas ke bawah. Struktur kontrol percabangan digunakan ketika program perlu mengambil keputusan berdasarkan suatu kondisi tertentu. Sedangkan perulangan digunakan ketika suatu perintah atau kelompok perintah perlu dijalankan berulang kali.

Perulangan biasanya digunakan dalam beberapa kondisi berikut:

1. Melakukan suatu proses yang sama beberapa kali
   Contohnya menampilkan sebuah teks sebanyak sepuluh kali atau menghitung jumlah dari beberapa angka secara berulang.
2. Melakukan proses secara bertahap dengan langkah yang sama
   Misalnya dalam proses perhitungan bertingkat atau pengolahan data secara berurutan, di mana setiap langkah memiliki pola yang sama tetapi menggunakan data yang berbeda.
3. Mengakses kumpulan data dalam suatu struktur data
   Perulangan juga sering digunakan untuk membaca atau memproses data yang terdapat dalam struktur data seperti List, Tuple, Queue, Stack, dan berbagai struktur data lainnya.

Dalam bahasa pemrograman, perulangan dapat dilakukan dengan beberapa cara, yaitu menggunakan perulangan for, perulangan while, maupun menggunakan teknik rekursif. Namun pada materi ini, pembahasan difokuskan pada penggunaan perulangan for dan while, karena kedua jenis perulangan tersebut merupakan metode yang paling umum digunakan dalam pemrograman Python.

MATERI 2 : Bentuk Perulangan for

Perulangan for umumnya digunakan ketika jumlah perulangan sudah diketahui sebelumnya atau ketika program perlu melakukan operasi yang sama pada sekumpulan data atau pada suatu rentang nilai tertentu.

Secara umum, perulangan for digunakan pada beberapa kondisi berikut.

Pertama, ketika jumlah perulangan telah diketahui sejak awal. Sebagai contoh, ketika program harus membaca data dari beberapa file teks. Misalnya terdapat sepuluh file teks yang harus dibaca satu per satu. Walaupun isi setiap file berbeda, proses membaca file tetap dilakukan dengan langkah yang sama. Oleh karena itu, proses tersebut dapat dilakukan menggunakan perulangan for dari file pertama hingga file ke-sepuluh.

Kedua, ketika program melakukan operasi yang sama pada suatu rentang nilai. Contohnya ketika menghitung jumlah dari bilangan 1 sampai 100. Program akan melakukan proses penjumlahan secara berulang dari angka 1, kemudian 2, 3, dan seterusnya sampai angka 100. Proses tersebut merupakan contoh penggunaan perulangan dalam rentang nilai tertentu.

Dalam Python, perulangan for biasanya digunakan bersama dengan fungsi range(). Fungsi ini digunakan untuk menghasilkan rentang bilangan yang akan diproses dalam perulangan. Fungsi range() memiliki beberapa bentuk penggunaan, yaitu:

1. range(stop)
   Digunakan untuk menghasilkan bilangan dari 0 sampai stop-1.
   Sebagai contoh, range(6) akan menghasilkan deret bilangan 0, 1, 2, 3, 4, 5.
2. range(start, stop, step)
   Digunakan untuk menghasilkan rentang bilangan mulai dari start sampai stop-1 dengan kenaikan nilai sebesar step. Jika nilai step tidak dituliskan, maka secara otomatis bernilai 1.

Contoh penggunaan perulangan for untuk menampilkan bilangan dari 1 sampai 100:

```
for i in range(1, 101):
  print(i)
```

Pada program tersebut, fungsi range(1, 101) menghasilkan bilangan mulai dari 1 sampai 100. Variabel i digunakan sebagai counter atau penghitung yang akan berubah nilainya pada setiap perulangan. Nilai i akan bertambah satu demi satu sesuai dengan nilai yang dihasilkan oleh fungsi range().

Jika program hanya membutuhkan perulangan tanpa menggunakan nilai penghitung, maka dapat digunakan tanda garis bawah (_) sebagai pengganti variabel counter. Contohnya sebagai berikut:

```
for _ in range(1, 101):
    print("Hello World")
```

Program tersebut akan menampilkan tulisan **"Hello World"** sebanyak 100 kali tanpa memanfaatkan nilai penghitung dari perulangan.

Selain itu, fungsi range() juga dapat menggunakan nilai langkah (step) yang berbeda. Misalnya untuk menampilkan bilangan genap dari 2 sampai 100:

```
for i in range(2, 101, 2):
    print(i)
```

Pada contoh tersebut, perulangan dimulai dari angka 2 dan bertambah 2 setiap langkah, sehingga bilangan yang dihasilkan adalah 2, 4, 6, 8, 10, dan seterusnya hingga 100.

Selain langkah positif, fungsi range() juga dapat menggunakan step negatif untuk menghasilkan urutan bilangan secara menurun. Contohnya untuk menampilkan bilangan genap dari 100 sampai 2:

```
for i in range(100, 1, -2):
    print(i)
```

Pada program tersebut, perulangan dimulai dari angka 100 dan berkurang 2 setiap langkah, sehingga menghasilkan deret bilangan 100, 98, 96, 94, dan seterusnya hingga 2.

MATERI 3 : Bentuk Perulangan While

Perulangan while biasanya digunakan ketika jumlah perulangan belum diketahui sebelumnya. Artinya, perulangan akan terus dilakukan selama suatu kondisi tertentu masih bernilai True. Jika kondisi tersebut berubah menjadi False, maka perulangan akan berhenti.

Secara umum, bentuk dasar perulangan while dapat dituliskan sebagai berikut:

```
start
while kondisi:
    operasi
    operasi
    step (opsional)
```

Penjelasan dari struktur tersebut adalah sebagai berikut:

- **start** merupakan nilai awal dari variabel yang digunakan dalam perulangan.
- **while kondisi** merupakan syarat yang menentukan apakah perulangan masih akan dijalankan atau tidak.
- **operasi** merupakan perintah-perintah yang dijalankan selama kondisi masih terpenuhi.
- **step** merupakan perubahan nilai variabel yang digunakan untuk mengontrol jalannya perulangan.
- Selama kondisi pada while bernilai True, maka program akan terus menjalankan perintah di dalam blok perulangan.

Contoh Penggunaan Perulangan while :

contoh program yang menggunakan perulangan while untuk memastikan bahwa pengguna memasukkan bilangan genap.

```
bilangan = 0
genap = False
while genap == False:
   bilangan = int(input('Masukkan bilangan genap: '))
   if bilangan % 2 == 0:
      genap = True
print(bilangan, 'yang anda masukkan adalah bilangan genap')
```

Penjelasan Program:

1. Inisialisasi variable
    - Variabel bilangan diberi nilai awal 0.
    - Variabel genap diberi nilai False, yang menandakan bahwa bilangan genap belum dimasukkan oleh pengguna.
2. Perulangan while
   Program menjalankan perulangan selama nilai genap masih False. Hal ini berarti program akan terus meminta pengguna memasukkan angka.
3. Input User
   Program meminta pengguna memasukkan sebuah bilangan menggunakan fungsi input().
4. Pengecekan bilangan genap
   Program memeriksa apakah bilangan tersebut merupakan bilangan genap dengan menggunakan operasi modulus %. (bilangan % 2 == 0) Jika kondisi tersebut terpenuhi, maka variabel genap diubah menjadi True.
5. Menghentikan perulangan
   Ketika variabel genap berubah menjadi True, kondisi pada while tidak lagi terpenuhi sehingga perulangan berhenti.
6. Menampilkan hasil
   Program kemudian menampilkan pesan bahwa bilangan yang dimasukkan merupakan bilangan genap.

MATERI 4 : Penggunaan Break dan Continue

Perulangan tidak hanya berjalan secara otomatis sesuai kondisi yang telah ditentukan, tetapi juga dapat dikontrol menggunakan perintah tertentu. Dua perintah yang sering digunakan untuk mengontrol jalannya perulangan adalah break dan continue.

Perintah break digunakan untuk menghentikan perulangan secara langsung, meskipun kondisi perulangan sebenarnya masih terpenuhi. Ketika program menemukan perintah break, maka perulangan akan segera dihentikan dan program akan melanjutkan eksekusi ke baris kode setelah perulangan tersebut.

Sedangkan perintah continue digunakan untuk melewati satu iterasi perulangan dan langsung melanjutkan ke iterasi berikutnya tanpa menjalankan sisa perintah yang berada di dalam blok perulangan pada iterasi tersebut.

Contoh Perulangan Tanpa Break dan Continue:

```
for i in range(1, 11):
  print(i)

print('Selesai')
```

Pada program tersebut, fungsi range(1, 11) menghasilkan bilangan dari 1 sampai 10. Variabel i digunakan sebagai penghitung dalam perulangan. Program akan mencetak setiap nilai i hingga mencapai angka 10, kemudian setelah perulangan selesai akan menampilkan tulisan "Selesai".

Penggunaan Break :

Jika diinginkan perulangan yang seharusnya berjalan sampai 10 dihentikan ketika mencapai angka tertentu, maka dapat digunakan perintah break. Maka:

```
for i in range(1, 11):
  if i == 5:
    break
  else:
    print(i)

print('Selesai')
```

Penjelasan:

1. Program melakukan perulangan dari angka 1 sampai 10.
2. Pada setiap iterasi, program memeriksa apakah nilai i sama dengan 5.
3. Ketika nilai i mencapai 5, kondisi i == 5 bernilai True.
4. Program kemudian menjalankan perintah break sehingga perulangan langsung dihentikan.
5. Setelah perulangan berhenti, program melanjutkan ke baris berikutnya dan menampilkan tulisan "Selesai".

Penggunaan Continue :

Perintah continue tidak menghentikan perulangan, tetapi hanya melewati iterasi tertentu dan langsung melanjutkan ke iterasi berikutnya. Contohnya :

```
for i in range(1, 11):
  if i == 6:
    continue
  else:
    print(i)

print('Selesai')
```

Penjelasan:

1. Program melakukan perulangan dari angka 1 sampai 10.
2. Setiap kali perulangan berjalan, program memeriksa apakah nilai i sama dengan 6.
3. Jika kondisi tersebut terpenuhi, program menjalankan perintah continue.
4. Akibatnya, angka 6 tidak ditampilkan dan program langsung melanjutkan ke iterasi berikutnya.
5. Setelah perulangan selesai, program menampilkan tulisan "Selesai".
6. Output yang dihasilkan adalah angka 1 sampai 10, tetapi angka 6 dilewati.

MATERI 5 : Konversi dari Bentuk for Menjadi Bentuk while

Perulangan dapat dilakukan dengan menggunakan beberapa cara, di antaranya perulangan for dan perulangan while. Pada dasarnya, sebagian besar perulangan for dapat diubah atau dikonversi menjadi perulangan while. Hal ini karena kedua jenis perulangan tersebut memiliki konsep dasar yang hampir sama.

Baik perulangan for maupun while, keduanya membutuhkan beberapa komponen penting agar perulangan dapat berjalan dengan benar. Komponen-komponen tersebut antara lain:

1. Nilai awal (initial value)
2. Nilai akhir (kondisi berhenti)
3. Langkah perubahan nilai (step)

Ketiga komponen tersebut harus ada agar proses perulangan dapat berjalan secara teratur dari awal hingga mencapai batas akhir.

Contoh Perulangan Menggunakan for:

```
for i in range(1, 11):
  print(i)
```

Pada contoh tersebut, dapat diidentifikasi beberapa komponen penting dari perulangan:

- Nilai awal = 1
- Nilai akhir = 10
- Langkah (step) = 1

Fungsi range(1, 11) menghasilkan bilangan dari 1 sampai 10, dengan kenaikan nilai setiap iterasi sebesar 1.

Konversi Perulangan for Menjadi while :
Perulangan for diatas dapat dikonversi menjadi perulangan while dengan cara mendefinisikan nilai awal terlebih dahulu, kemudian menentukan kondisi perulangan, serta menambahkan perubahan nilai pada setiap iterasi.

Contoh konversinya:

```
i = 1   # nilai awal

while i <= 10:   # kondisi akhir
   print(i)
   i = i + 1   # step
```

Penjelasan:

1. Inisialisasi nilai awal
   Variabel i diberi nilai awal 1 sebagai titik awal perulangan.
2. Kondisi perulangan
   Perulangan akan terus berjalan selama nilai i masih kurang dari atau sama dengan 10.
3. Menjalankan operasi
   Program mencetak nilai i pada setiap iterasi.
4. Perubahan nilai (step)
   Nilai i ditambahkan 1 setiap kali perulangan berlangsung sehingga nilai i secara bertahap meningkat sampai mencapai batas akhir.

Ketika nilai i sudah lebih dari 10, kondisi while menjadi False, sehingga perulangan berhenti.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

Buatlah program yang menerapkan perhitungan perkalian dengan menggunakan penjumlahan. Buatlah fungsi perkalian() dalam program tersebut! Berikut ini adalah beberapa contoh perhitungan yang diharapkan:

- 6 x 5 = 5 + 5 + 5 + 5 + 5 + 5 = 30.
- 7 x 10 = 10 + 10 + 10 + 10 + 10 + 10 + 10 = 70.

soal01.py

```python
def perkalian(a, b):
    hasil = 0

    for i in range(a):
        hasil = hasil + b
        if i < a-1:
            print(b, end=" + ")
        else:
            print(b, end="")

    print(" =", hasil)


# Contoh
perkalian(6, 5)
perkalian(7, 10)
```

- Fungsi perkalian(a, b) dibuat untuk menerima dua bilangan yang akan dikalikan.
- Membuat variabel hasil yang digunakan untuk menyimpan jumlah dari penjumlahan yang dilakukan secara berulang.
- Bagian Perulangan for digunakan sebanyak nilai a, karena angka b akan dijumlahkan sebanyak a kali.
- Pada setiap proses iterasi, nilai b ditambahkan ke variabel hasil.

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS E:\Prak_Alpro\minggu06> & "C:\Users\ALFA DELTA\AppData\Local\Prog
5 + 5 + 5 + 5 + 5 + 5 = 30
10 + 10 + 10 + 10 + 10 + 10 + 10 = 70
PS E:\Prak_Alpro\minggu06>
```

SOAL 2

Buatlah program yang dapat menampilkan deret bilangan ganjil dari batas bawah dan batas atas yang diberikan oleh pengguna. Jika ternyata batas atas < batas bawah, berarti deret tersebut dimulai dari batas atas, sampai batas bawah (negatif range). Buatlah fungsi ganjil() dalam program tersebut! Berikut ini adalah contoh hasil yang diharapkan:

• bawah = 10, atas = 30. Karena bawah < atas, berarti dari kecil ke besar, maka hasilnya adalah: 11, 13, 15, 17, 19, 21, 23, 25, 27, 29.

• bawah = 97, atas = 82. Karena bawah > atas, berarti dari besar ke kecil, maka hasilnya adalah: 97, 95, 93, 91, 89, 87, 85, 83.

```python
soal02.py
def ganjil(bawah, atas):
    if bawah < atas:
        for i in range(bawah, atas + 1):
            if i % 2 != 0:
                print(i, end=", ")
    else:
        for i in range(bawah, atas - 1, -1):
            if i % 2 != 0:
                print(i, end=", ")

# Input
bawah = int(input("Masukkan batas bawah: "))
atas = int(input("Masukkan batas atas: "))

ganjil(bawah, atas)
```

- Fungsi ganjil(bawah, atas) dibuat untuk menerima dua parameter yaitu batas bawah dan batas atas.
- Menggunakan percabangan if untuk menentukan arah perulangan:
    - Jika bawah < atas, maka perulangan dilakukan dari kecil ke besar.

    - Jika bawah > atas, maka perulangan dilakukan dari besar ke kecil.
- Perulangan for digunakan untuk menelusuri setiap angka dalam rentang yang diberikan.
- Setiap angka diperiksa menggunakan kondisi: i % 2 != 0
  Jika memenuhi kondisi tersebut, angka akan ditampilkan.

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS E:\Prak_Alpro\minggu06> & "C:\Users\ALFA DELTA\AppData
Masukkan batas bawah: 10
Masukkan batas atas: 30
11, 13, 15, 17, 19, 21, 23, 25, 27, 29,
PS E:\Prak_Alpro\minggu06> & "C:\Users\ALFA DELTA\AppData
Masukkan batas bawah: 97
Masukkan batas atas: 82
97, 95, 93, 91, 89, 87, 85, 83,
PS E:\Prak_Alpro\minggu06>
```

SOAL 3

Buatlah sebuah program penghitung nilai Indeks Prestasi Semester (IPS). Input bagi program:

• Jumlah mata kuliah

• Nilai A, B, C, dan D untuk setiap mata kuliah mahasiswa.

Diasumsikan sks setiap mata kuliah selalu 3. Kemudian bobot dari masing-masing nilai adalah: A=4, B=3, C=2, D=1.

Output program ialah hasil IPS yang didapatkan. Jalannya program seperti pada Gambar 6.7.

soal03.py

```python
print("Program penghitung IPS Mahasiswa")

jumlah_mk = int(input("Berapa jumlah mata kuliah: "))

total_bobot = 0
sks = 3

for i in range(1, jumlah_mk + 1):
    nilai = input(f"Nilai MK {i}: ").upper()

    if nilai == "A":
        bobot = 4
    elif nilai == "B":
        bobot = 3
    elif nilai == "C":
        bobot = 2
    elif nilai == "D":
        bobot = 1
    else:
        bobot = 0

    total_bobot += bobot * sks

ips = total_bobot / (jumlah_mk * sks)

print("Nilai IPS anda semester ini", round(ips, 2))
```

- Menampilkan judul, *print("Program penghitung IPS Mahasiswa")*. Perintah untuk menampilkan teks Program penghitung IPS Mahasiswa.
- jumlah_mk = int(input("Berapa jumlah mata kuliah: ")). Program meminta Nilai yang dimasukkan oleh user akan disimpan dalam variabel jumlah_mk dan diubah menjadi tipe data integer karena akan digunakan dalam proses perulangan.
- Inisialisasi Variabel
    - total_bobot digunakan untuk menyimpan total hasil perkalian bobot nilai dengan SKS.
    - sks berisi jumlah SKS setiap mata kuliah yang mana diasumsikan selalu bernilai 3.
- Perulangan for untuk meminta pengguna memasukkan nilai setiap mata kuliah. Perulangan akan berjalan sebanyak jumlah mata kuliah yang dimasukkan. Variabel **i** digunakan sebagai penghitung sehingga program dapat menampilkan urutan mata kuliah seperti Nilai MK 1, Nilai MK 2, dan seterusnya. Fungsi upper**()** digunakan agar input nilai huruf tetap dikenali walaupun pengguna mengetik huruf kecil.
- Setelah nilai dimasukkan, program menentukan bobot nilai menggunakan struktur percabangan if-elif. Setiap nilai huruf akan diberikan bobot sesuai dengan bobot nilai.
- Setelah itu, program akan menghitung total bobot dengan mengalikan bobot dengan jumlah SKS. Nilai tersebut kemudian dijumlahkan ke dalam variabel total_bobot.

- Setelah semua nilai mata kuliah dimasukkan, program menghitung nilai IPS menggunakan rumus, *ips = total_bobot / (jumlah_mk * sks)*
- Terakhir adalah menampilkan hasil perhitungan IPS kepada pengguna. Fungsi round(ips, 2) digunakan untuk membulatkan nilai IPS hingga dua angka di belakang koma, sehingga hasil yang ditampilkan lebih rapi.

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS E:\Prak_Alpro\minggu06> & "C:\Users\ALFA DELTA\AppData\
Program penghitung IPS Mahasiswa
Berapa jumlah mata kuliah: 6
Nilai MK 1: A
Nilai MK 2: B
Nilai MK 3: C
Nilai MK 4: A
Nilai MK 5: D
Nilai MK 6: C
Nilai IPS anda semester ini 2.67
PS E:\Prak_Alpro\minggu06>
```

Github : https://github.com/71200556/71200556_frans